Manon Clair
Maria Camila Cuta Sona

Rapport de Projet : Planning Poker

Conception Agile - M1 Informatique

---

1. Introduction et Contexte

Présentation du projet

L'application Planning Poker est un logiciel destiné à l'estimation collaborative et dynamique de tâches dans un contexte agile. Elle permet aux membres d'une équipe de proposer simultanément leurs estimations et d'aboutir à un consensus sur la complexité des tâches du backlog. Et grâce à ce logiciel on pourrait obtenir des estimations qui prennent en compte tous les intégrants d'une équipe de développement.

Objectif

Développer une application que permettre faire l'estimation des taches selon sa complexité, les spécifications du projet, mettant en place les modes de vote définis et respectant une architecture logicielle structurée.

Organisation du travail

Binôme : Manon Claire et Maria Camila Cuta

Méthodologie : Pair Programming

On a commencé pour faire la planification du projet en utilisant GitHub Projets ou on a créé un Kanban avec toutes les activités principales du projet en on las a classé en in progress, in review et done avec chaque responsable et une description. Après on a commencé le développement de l'application sous la méthodologie pair programming que on a implémente avec GitHub. On a défini deux rôles principaux pour le projet le premier c'était le driver, charge d'écrire le code et le deuxième le navigateur, responsable de relire le code en direct pour valider le code, et si c'est le cas proposer des améliorations. Dans tout le développement du projet les deux intégrants du projet ils ont échangé systématiquement ses deux rôles. Lien : https://github.com/users/mariacx-bit/projects/2

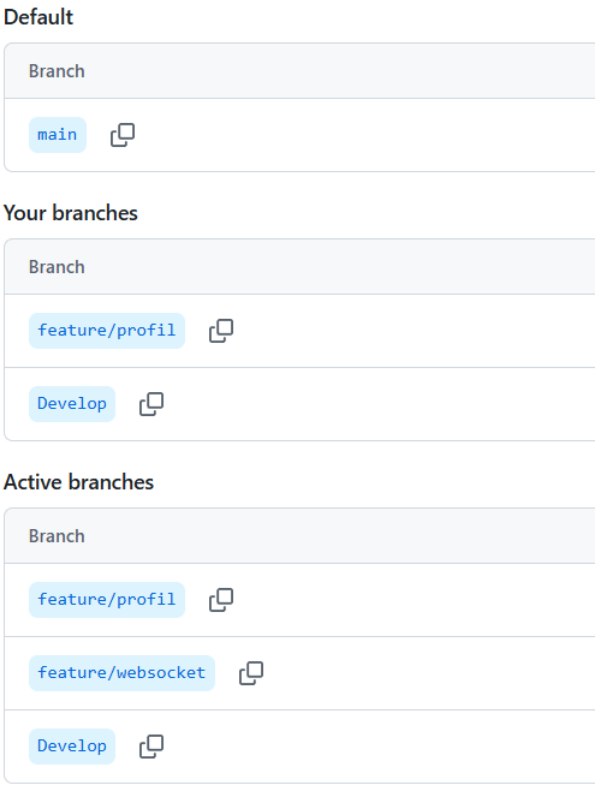
Stratégie : Branches

Pour faciliter le versioning et la lecture des différentes mises à jour, une stratégie de branches a été mise en place :

- Des branches feature/ : elles sont une copie de la branche principale et elles servent à développer des nouvelles fonctionnalités sans impacter le code de base. Il y a une branche par fonctionnalité.
- Une branche Develop : elle permet de fusionner toutes les fonctionnalités qui ont été créées dans les branches Feature/
- Une branche main : elle contient le code stable, validé et testé, prêt à être mis en production.

Branches :

Manon Clair
Maria Camila Cuta Sona



---

2. Choix Techniques et Architecture

2.1 Langage et Frameworks

Langage choisi :PHP / JavaScript

Nous avons choisi PHP pour implémenter le côté serveur afin de gérer la logique métier, le traitement des requêtes HTTP, la gestion des sessions utilisateurs ainsi que l'interaction avec la base de données. Ce choix s'explique par son intégration native avec les serveurs web et par sa capacité à générer du contenu HTML de manière dynamique.

Du côté client, afin d'assurer l'interactivité de l'application, nous avons choisi JavaScript, car il permet de mettre à jour dynamiquement l'interface graphique, notamment grâce à la gestion des interactions en temps réel. De plus, JavaScript facilite le développement d'applications web collaboratives et améliore l'expérience utilisateur.

Frameworks et bibliothèques principales :

Frameworks : Pour ce projet, nous avons choisi de développer l'application en PHP natif (côté serveur) et JavaScript natif (côté client) sans utiliser de framework applicatif. Cette décision s'appuie sur plusieurs considérations pédagogiques et techniques. D'une part, le développement natif permet une maîtrise directe des mécanismes fondamentaux du web, tels que la gestion des requêtes HTTP, les sessions et l'accès à la base de données, sans abstraction supplémentaire fournie par un framework. D'autre part, cela favorise une meilleure compréhension des principes sous-jacents des architectures web client–serveur, ce qui constitue un apprentissage essentiel avant l'usage de solutions plus complexes ou spécialisées. Enfin, l'ampleur fonctionnelle du projet Planning Poker ne nécessite pas les

fonctionnalités avancées que proposent les frameworks modernes, ce qui rend l'approche native suffisante, claire et suffisamment flexible pour répondre aux besoins du projet.

Bibliothèques : On a mis en place le fichier Composer, ce qui permet de déclarer les dépendances externes et d'assurer la gestion des packages via Composer. Dans ce cas, les dépendances étant minimales, l'utilisation de Composer garantit une organisation standardisée du code et facilite une éventuelle extension technique future. De plus, grâce à l'utilisation des bibliothèques natives de PHP, on peut également assurer une meilleure portabilité et une réduction des dépendances externes, ce qui simplifie la maintenance de l'application web.

Frontend : Pour le frontend, JavaScript natif est utilisé afin de gérer les interactions dynamiques de l'interface utilisateur, notamment l'envoi de requêtes asynchrones ou la mise à jour partielle de la page sans rechargement complet. Ce choix nous permet de maîtriser les interactions utilisateur à un niveau granulaire, ce qui est utile pour une application web interactive comme Planning Poker. De même, l'usage de HTML et CSS standard, et, lorsque nécessaire, d'un framework CSS léger comme Bootstrap, permet d'assurer une mise en page cohérente et responsive sans recourir à des bibliothèques JavaScript frontales complexes.

2.2 Architecture Logicielle

Approche architecturale : On a utilisé une architecture MVC, qui sépare les responsabilités du code en trois composants :

Modèle : Contient la logique métier et l'accès aux données. Les classes du modèle interagirent avec la base de données SQL pour récupérer, insérer ou mettre à jour les informations liées aux utilisateurs, aux sessions et aux parties de planning poker. Dans cette couche on garantit que toutes les règles métier sont centralisées et indépendantes de l'interface utilisateur.

Vue : Constitue la couche de présentation. Elle contient les fichiers HTML, CSS et JavaScript qui affichent l'interface utilisateur. La vue est responsable de l'affichage des informations fournies par le modèle et de la capture des interactions utilisateur, comme les votes ou les mises à jour en temps réel.

Contrôleur : Agit comme un intermédiaire entre le modèle et la vue. Le contrôleur reçoit les requêtes HTTP, appelle les méthodes appropriées du modèle, et sélectionne la vue à afficher. Il gère également la logique de navigation et le traitement des actions utilisateurs, permettant de cette manière une interaction fluide entre le client et le serveur.

WebSocket communication en temps réel : Pour garantir une expérience réellement collaborative, nous avons mis en place un canal de communication en temps réel basé sur un WebSocket entre le serveur PHP et les navigateurs des joueurs. Lorsqu'un participant rejoint une partie, modifie son vote ou qu'un consensus est calculé pour une tâche, le serveur envoie immédiatement un message sur le WebSocket, ce qui permet de mettre à jour l'interface de tous les clients sans rechargement de page. Cette approche évite les rafraîchissements périodiques (polling HTTP), réduit la latence perçue et assure que tous les joueurs voient en même temps l'état de la session (votes reçus, révélations de cartes, changement de tâche), ce qui est essentiel pour simuler une vraie séance de planning poker en présentiel.

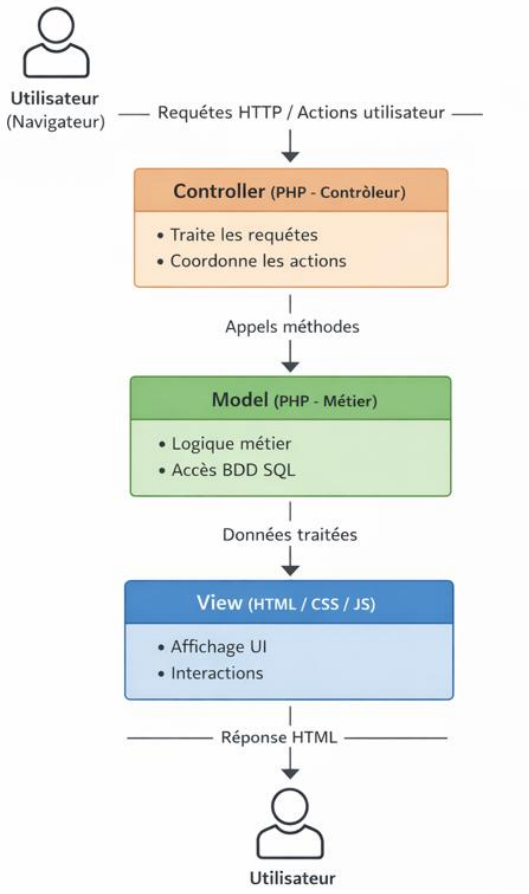
Cette organisation du code nous a permis structurer l'application de manière claire et modulaire, facilitant la maintenance, l'évolution et le développement du projet.

Avantages de cette architecture :

- Séparation des responsabilités **:** Chaque couche a une fonction distincte, grâce à ça on peut réduit les dépendances et rend le code plus clair et plus facile à maintenir.

- Réutilisabilité : On a créé les modèles et contrôleurs pourque ils puissent être réutilisés dans différentes parties de l'application, ou étendus pour ajouter de nouvelles fonctionnalités sans modifier la vue.
- Facilité d'évolution et de maintenance : Les changements dans l'interface utilisateur n'affectent pas la logique métier et inversement, ce qui rend le projet plus flexible et moins sujet aux erreurs lors des mises à jour.
- Adapté aux projets collaboratifs : La structure MVC est optimale pour travailler sur méthodologies comme pair programming ce que permet à plusieurs développeurs de travailler simultanément sur différentes couches (front-end, back-end) sans conflit.

Schéma de l'architecture MVC – PlanningPoker



2.3 Modélisation et Diagrammes

Entités principales du domaine :

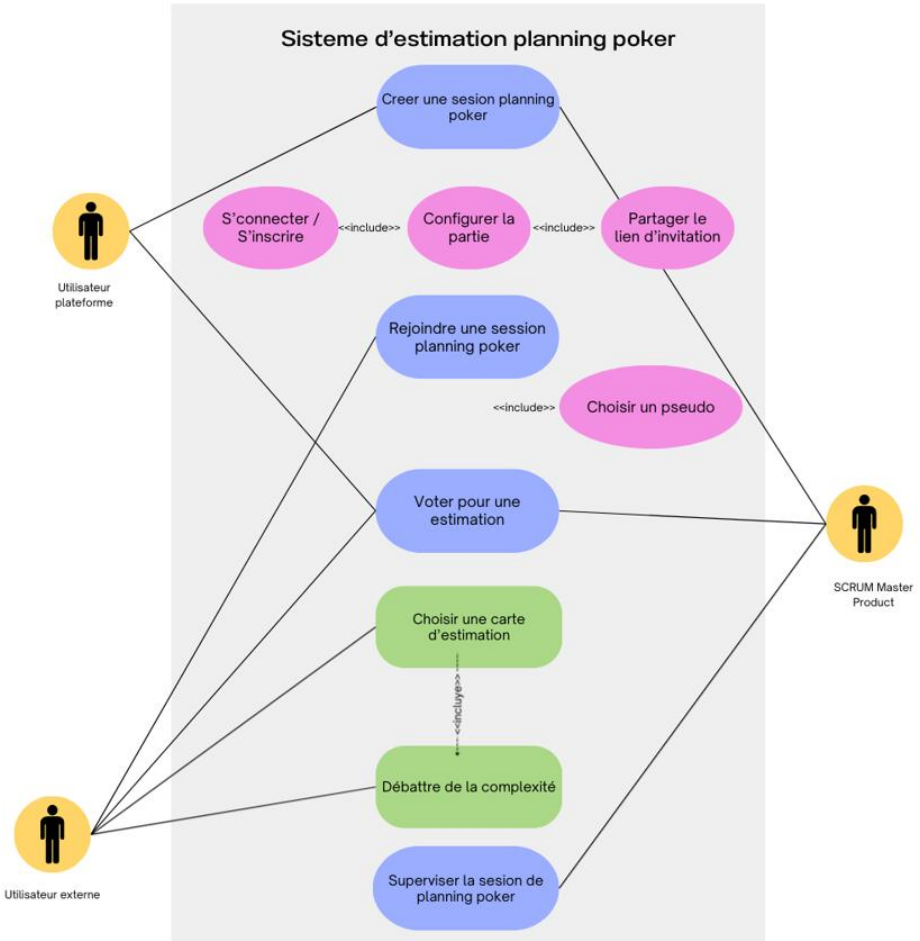
- **Joueur** : Le joueur représente un participant à une session de Planning Poker. Il peut être un utilisateur authentifié ou un utilisateur externe rejoignant une partie via un lien d'invitation. Un joueur est identifié par un pseudonyme et peut avoir un rôle spécifique au sein de la partie (par exemple, maître de la session). Cette entité est matérialisée dans le modèle par la table JoueurPartie, qui permet de gérer la participation des utilisateurs à une session donnée.
- **Backlog** : Le backlog correspond à l'ensemble des tâches à estimer au cours d'une session de Planning Poker. Dans le modèle de données, le backlog est associé à une Partie et stocké sous

forme structurée (par exemple via un champ backlog_json). Il permet de centraliser les tâches à analyser et d'organiser le déroulement de la session.

- **Tâche** : Une tâche représente un élément du backlog nécessitant une estimation. Elle est définie par un identifiant, une description et un résultat correspondant à l'estimation finale. Une tâche est associée à une ou plusieurs parties et constitue l'unité centrale du processus d'estimation collective.
- **Vote** : Le vote correspond à l'estimation fournie par un joueur pour une tâche donnée lors d'une session. Les votes sont collectés afin de permettre la comparaison des estimations et de faciliter la discussion entre les participants. Les résultats des votes sont ensuite agrégés et stockés au niveau de la partie.
- **Règles** : Les règles définissent le cadre du déroulement d'une session de Planning Poker. Elles incluent notamment les modalités de vote, le choix des cartes d'estimation, l'ordre de traitement des tâches et la gestion des tours. Ces règles structurent les interactions entre les joueurs et garantissent la cohérence du processus d'estimation.

Diagramme de cas d'utilisation

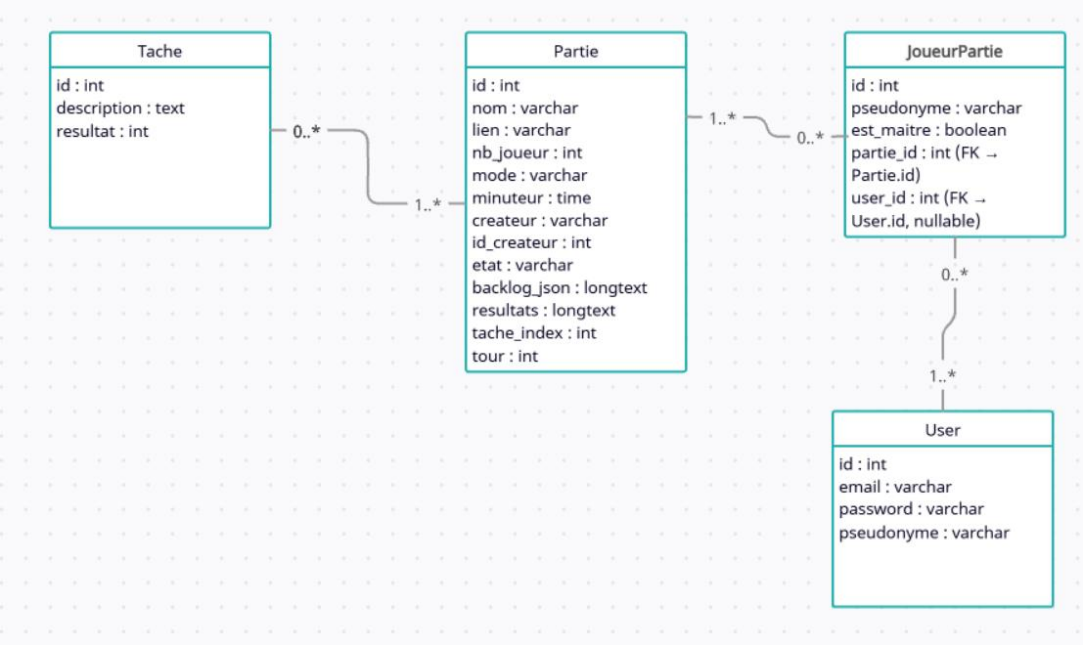
Le diagramme de cas d'utilisation met en évidence les principales fonctionnalités du système d'estimation Planning Poker.
Il identifie trois types d'acteurs : Utilisateur plateforme, Utilisateur externe et Scrum Master / Product Owner.



Manon Clair
Maria Camila Cuta Sona

Diagramme de classes

Le diagramme de classes UML présente les entités centrales du modèle de données et leurs relations



2.4 Gestion des Données

Format choisi : JSON

Dans notre application, les données métier sont stockées dans une base MariaDB, mais les échanges entre le front et le back, ainsi que la persistance du backlog et des résultats d'une partie, s'appuient sur des structures JSON sérialisées dans les colonnes backlog_json et résultats de la table partie.

Le format JSON a été choisi pour sa souplesse (facile à faire évoluer), sa bonne intégration avec les frameworks web et sa proximité avec les objets manipulés côté application (tableaux, dictionnaires).

Structure du backlog : Le backlog représente la liste des tâches à estimer, en cohérence avec la table tache (champs id, description, resultat).


```
{
 "game_id": 1,
 "tasks": [
   {
    "id": 1,
    "description": "Implémenter l'écran de connexion",
    "resultat": null
   },
   {
    "id": 2,
    "description": "Ajouter le mode de vote Fibonacci",
    "resultat": 5
   }
```

```
 ],
 "current_task_index": 0
}
```


- game_id : identifiant de la partie (clé primaire de la table partie).
- tasks : liste des tâches, chaque entrée reprenant l'id et la description de la table tache, ainsi que resultat qui stocke la valeur estimée finale (ou null si la tâche n'est pas encore estimée).
- current_task_index : index de la tâche en cours d'estimation, synchronisé avec le champ tache_index de la table partie.

Structure de sauvegarde de partie : La sauvegarde d'une partie regroupe l'état global de la session : configuration, votes effectués et consensus obtenu pour chaque tâche.


```
{
 "game_id": 1,
 "nom": "Session Planning Poker Sprint 3",
 "lien": "e504ca8c",
 "mode": "fibonacci",
 "nb_joueur": 5,
 "minuteur": "00:02:00",
 "etat": "en_cours",
 "tour": 1,
 "taches": [
  {
   "tache_id": 1,
   "description": "Implémenter l'écran de connexion",
   "votes": [
    { "joueur_id": 10, "pseudonyme": "Dev1", "estimate": 3 },
    { "joueur_id": 11, "pseudonyme": "Dev2", "estimate": 5 }
   ],
   "consensus": 5
  },
  {
   "tache_id": 2,
   "description": "Ajouter le mode de vote Fibonacci",
   "votes": [],
   "consensus": null
  }
 ]
}
```


- Les champs game_id, nom, lien, mode, nb_joueur, minuteur, etat, tour reflètent directement les colonnes de la table partie.
- Le tableau taches contient, pour chaque tâche, son identifiant tache_id, sa description, la liste des votes et le consensus calculé.
- Chaque vote inclut le joueur_id et le pseudonyme, ce qui permet de reconstruire l'historique des estimations même pour les joueurs non connectés (user_id nullable)

---

3. Mise en place de l'Intégration Continue

3.1 Description du Pipeline CI

Manon Clair
Maria Camila Cuta Sona

Plateforme utilisée : La plateforme d'intégration continue utilisée pour ce projet est GitHub, via un workflow défini dans le répertoire. github/workflows du dépôt PlanningPoker. GitHub est utilisé pour construire, tester et analyser automatiquement l'application PHP à chaque push ou pull request sur le dépôt mariacx-bit/PlanningPoker.

Le workflow est décrit dans un fichier YAML qui précise les déclencheurs, les jobs, l'image de base (par exemple ubuntu-latest) et les versions de PHP ou de Node utilisées

Workflow général :

1. **Checkout du code :** Le job commence par l'action actions/checkout, qui récupère l'intégralité du code du dépôt (application, tests, configuration). Cette étape garantit que la version testée correspond exactement au commit poussé sur GitHub, y compris les fichiers de configuration (composer.json, .env.test, etc.).
2. **Installation des dépendances :** Le pipeline installe d'abord l'environnement PHP puis exécute composer install --no-interaction --no-progress pour récupérer les dépendances backend. Si le front est utilisé, les dépendances JavaScript peuvent être installées via npm install ou yarn install, ce qui permet de construire les assets nécessaires à l'interface de Planning Poker.
3. **Exécution des tests unitaires :** Une fois les dépendances installées, GitHub Actions lance la suite de tests PHPUnit, typiquement avec une commande du type php bin/phpunit sur l'environnement de test. Ces tests valident la logique métier principale : création de partie, gestion des tâches, calcul des résultats d'estimation, droits des joueurs, etc., afin d'éviter les régressions avant le merge.
4. **Génération de la documentation technique :** Le workflow peut inclure une étape de génération de documentation à partir des commentaires du code PHP produisant une sortie HTML.
5. **Archivage des résultats :** Les rapports de tests (logs, éventuel rapport JUnit ou couverture de code) et la documentation générée sont enregistrés comme artefacts GitHub via l'action actions/upload-artifact. Ces artefacts sont accessibles depuis l'interface GitHub pour permettre à l'enseignant de consulter les résultats de la CI (succès/échec des tests, documentation) sans avoir à relancer localement le pipeline.

3.2 Tests Unitaires

Framework de test utilisé : PHPUnit

Couverture des tests :

- **Calcul de consensus** : Des tests vérifient que, pour un ensemble de votes, la valeur de consensus retournée est correcte selon le mode sélectionné (moyenne, médiane, majorité, fibonacci/strict).
- **Validation de l'unanimité** : Des tests spécifiques couvrent la règle du premier tour à l'unanimité : si tous les joueurs votent la même valeur, le système doit détecter l'unanimité et arrêter le tour.
- **Gestion des votes :** Des tests valident que l'enregistrement d'un vote ne peut pas se faire pour un joueur inexistant ou une tâche inexistante (cohérence avec les tables user, joueur_partie, tache). Ils vérifient également qu'un même joueur ne peut pas voter plusieurs fois pour la même tâche, et que les valeurs envoyées respectent l'échelle autorisée (par exemple la suite Fibonacci)
- **Chargement des données** : Des tests garantissent que le backlog et les résultats stockés dans backlog_json et résultats sont correctement parses en objets applicatifs, avec présence

des champs obligatoires (tache_id, description, votes, consensus. En cas de JSON invalide ou de champ manquant, la logique de validation doit lever une erreur ou rejeter la donnée, ce qui est vérifié par des tests unitaires d'exception.

3.3 Génération de Documentation

Outil utilisé : phpDocumentor

- phpDocumentor analyse le code PHP (contrôleurs, entités, services) et les commentaires structurés pour produire une API reference complète.
- L'outil s'intègre facilement au workflow existant (exécution via une simple commande phpdoc run -d src -t docs/api dans le pipeline GitHub Actions)

Contenu généré :

- Documentation des classes et fonctions : pour chaque classe (par exemple services de calcul de consensus, entités Partie, Tache, JoueurPartie), phpDocumentor génère des pages détaillant attributs, méthodes, paramètres et valeurs de retour.
- Guides d'utilisation technique : des fichiers Markdown ou reStructuredText ajoutés au dépôt (par exemple docs/installation.md, docs/usage-planning-poker.md) sont intégrés à la sortie pour décrire l'installation, la configuration et les scénarios d'utilisation.
- Diagrammes UML : phpDocumentor peut produire des diagrammes de classes à partir du code, grâce à l'intégration avec PlantUML ou GraphViz, ce qui complète les diagrammes UML conçus pour le rapport.
- Référence d'API : l'ensemble des namespaces et services de l'application est regroupé dans une navigation hiérarchique (par paquet / répertoire), jouant le rôle de référence d'API pour les développeurs.

Accessibilité :

La documentation technique est générée automatiquement avec phpDocumentor dans la CI. Elle couvre les classes métier, les services de calcul de consensus, les entités de base de données et inclut des diagrammes UML. Le résultat est publié en artefact GitHub Actions et, pour une consultation facile, exposé via GitHub Pages

---

4. Manuel Utilisateur et Fonctionnalités

4.1 Installation et Lancement

Prérequis :

- PHP 8.x installé sur la machine (via XAMPP, Wamp, MAMP ou PHP en ligne de commande).
- Serveur web pouvant servir du PHP (Apache ou serveur interne de PHP) et une base MariaDB/MySQL avec la base PlanningPoker importée

Étapes d'installation :

Cloner le repository

git clone https://github.com/mariacx-bit/PlanningPoker.git

Manon Clair
Maria Camila Cuta Sona

cd PlanningPoker

Configurer la base de données

- Créer la base PlanningPoker dans MariaDB/MySQL.
- Importer le fichier SQL fourni (par exemple PlanningPoker-2-1.sql) via phpMyAdmin ou la ligne de commande : mysql -u <user> -p PlanningPoker < PlanningPoker-2-1.sql
- Vérifier/adapter le fichier de configuration PHP (par exemple config.php ou db.php) pour y mettre l'hôte, le nom de base, l'utilisateur et le mot de passe

Lancer l'application

Depuis un serveur Apache/PHP classique :

- Copier le dossier du projet dans le répertoire web (par ex. htdocs/PlanningPoker sous XAMPP).
- Accéder à l'application via le navigateur : http://localhost/PlanningPoker
- Ou avec le serveur interne de PHP, depuis la racine du projet : php -S 127.0.0.1:8000
- Puis ouvrir http://127.0.0.1:8000 dans le navigateur.

4.2 Modes de Jeu

Mode Strict :

- Dans le mode strict, chaque joueur choisit librement une valeur d'estimation dans la série de cartes proposée (par exemple suite de Fibonacci ou valeurs prédéfinies), sans contrainte de borne autre que celles définies par l'interface.
- À la fin du tour, toutes les cartes sont révélées simultanément et l'application applique directement la règle de consensus configurée pour la partie (moyenne, médiane ou majorité) à l'ensemble des votes enregistrés pour la tâche courante.

Mode complémentaire Moyenne:

- Dans le mode moyenne, le consensus affiché pour une tâche correspond à la moyenne arithmétique des valeurs choisies par les joueurs, arrondie à l'entier le plus proche pour être compatible avec le champ résultat de la table tache.
- Ce mode est particulièrement adapté lorsque l'équipe veut lisser les écarts d'estimation : des votes extrêmes influencent le résultat, mais sans imposer automatiquement la valeur la plus élevée ou la plus basse

Règle du premier tour :

- Lors du premier tour de vote pour une tâche, l'application vérifie si toutes les estimations saisies par les joueurs sont identiques. Si c'est le cas, cette valeur est directement enregistrée

comme consensus dans le champ résultat de la tâche, sans lancer de nouveau tour de discussion.

- Si au contraire les votes diffèrent, aucun consensus n'est enregistré et l'interface propose un échange entre les joueurs, puis éventuellement un nouveau tour de vote sur la même tâche jusqu'à stabilisation d'un consensus via la règle du mode choisi (moyenne dans notre implémentation).

4.3 Interface Utilisateur

Flux principal d'une partie :

1. **Menu d'accueil** : L'utilisateur choisit un pseudonyme, le nombre de joueurs et le mode de jeu (strict + mode de consensus choisi) avant de créer une nouvelle partie, ce qui initialise un enregistrement dans la table partie.
2. **Chargement du Backlog** : Le créateur importe un fichier JSON contenant la liste des tâches, qui est parsée puis stockée dans la colonne backlog_json de la partie, en cohérence avec la table tache.
3. **Phase de vote** : Pour chaque tâche active (tache_index), les joueurs connectés (table joueur_partie) voient la description et saisissent leur estimation ; les votes sont enregistrés en base ou dans la structure JSON de résultats.
4. **Affichage du consensus** : Une fois tous les votes reçus, l'application calcule le consensus (règle du mode sélectionné) et affiche la valeur retenue ainsi que, éventuellement, la moyenne brute des votes.
5. **Sauvegarde de la partie** : À la fin de la session, l'état de la partie est sérialisé (backlog, résultats, tour, index de tâche) dans les champs backlog_json et resultats, avec possibilité d'exporter un fichier JSON de sauvegarde.

Caractéristiques principales :

- Gestion multi-joueurs en ligne : plusieurs utilisateurs peuvent rejoindre une même partie via un lien ou un code, chaque joueur étant représenté par un enregistrement dans joueur_partie avec son pseudonyme et son rôle (maître ou non).
- Support de plusieurs modes de consensus : le système permet de configurer le mode de calcul (moyenne, médiane, majorité, strict), appliqué automatiquement à la fin de chaque tour de vote.
- Persistance complète des parties : le backlog et l'historique des votes/consensus sont sauvegardés en JSON dans la table partie, ce qui permet de reprendre ou analyser une session ultérieurement.

Mise en ligne : on a hébergé le site sur OVH

URL: https://fiid/planning-poker

---

5. Points de Vigilance et Respect des Consignes

Vérifications effectuées

- Fichier README : Un fichier README est présent à la racine du dépôt et décrit l'installation, le lancement et les principaux usages de l'application de planning poker.

- Format JSON : Le backlog et les sauvegardes de parties utilisent un format JSON structuré et validé (tâches, votes, consensus) avant enregistrement dans les colonnes backlog_json et resultats de la table partie.
- Règles de vote : Les règles de vote implémentées (mode strict, calcul de consensus par moyenne/médiane/majorité et règle du premier tour unanime) respectent les spécifications données dans le sujet de projet.
- Repository propre : Le dépôt ne contient que le code source et les fichiers utiles (PHP, SQL, assets), sans fichiers compilés ni environnements virtuels lourds, afin de faciliter le clonage et la revue.
- Orthographe et documentation : Le rapport, le README et les commentaires de code ont été relus pour corriger les fautes majeures et offrir une documentation claire pour l'évaluateur et les futurs contributeurs.

État du projet

- Fonctionnalités complètement implémentées : création et gestion de parties (configuration du nombre de joueurs, mode de jeu, minuteur), gestion des utilisateurs/joueurs (user, joueur_partie), stockage des tâches et des résultats dans la base PlanningPoker et en JSON (backlog_json, resultats), ainsi que le flux principal de vote avec calcul de consensus.
- Fonctionnalités en cours / partielles : interface utilisateur encore perfectible (ergonomie, affichage détaillé de l'historique des tours), couverture de tests unitaires à renforcer sur certains cas limites (votes manquants, annulation de vote, tâches sans backlog initial).
- Améliorations futures envisagées : ajout de statistiques sur les parties (temps moyen d'estimation, répartition des scores), amélioration de la gestion des rôles (distinction plus nette maître de jeu/joueurs), internationalisation de l'interface et déploiement automatisé depuis la CI vers un hébergement en ligne.

Conclusion

Ce projet de planning poker a permis de concevoir et développer une application web de gestion d'estimations agiles, depuis la modélisation des données jusqu'à la mise en place d'un fonctionnement complet de session avec différents modes de consensus et une communication en temps réel via WebSocket.

La principale difficulté a résidé dans la gestion correcte des règles métier (calcul de consensus, premier tour unanime, persistance en JSON), dans la mise en œuvre fiable de la synchronisation temps réel entre les navigateurs des joueurs, ainsi que dans la coordination du travail en binôme avec une intégration continue et des tests unitaires.

Ce travail a permis d'approfondir la pratique du développement collaboratif, la structuration d'un projet PHP natif, l'utilisation d'une base MariaDB, la mise en place d'un canal WebSocket temps réel et la formalisation de la documentation technique et utilisateur, compétences réutilisables dans de futurs projets.